Q1) Using MPLAB for the PIC18F4550 microcontroller, write an Assembly language program to load two 8-bit values into working registers, add them, store the result in data memory, and observe the WREG, STATUS register, and result location during debugging.

```
PROCESSOR 18F4550
#include <xc.inc>

PSECT mycode
   MOVLW 0x25  //Access Bank addressing
   MOVWF 0x20
   MOVLW 0x35
   ADDWF 0x20 ,W //Defaults to F
   MOVWF 0x21
HERE :
   BRA HERE
END
```

Q2) Using MPLAB for the PIC18F4550 microcontroller, write an Assembly language program to subtract two 8-bit numbers and observe the result along with the relevant STATUS register flags in the simulator/debugger.

```
PRCOCESSOR 18F4550
#include <xc.in>
PSECT mycode
        MOVLW 0x25
        MOVWF 0x20
        MOVLW 0x35
        SUBWF 0x20 ,W
```

Q3) Using MPLAB for the PIC18F4550 microcontroller, write an Assembly language program to transfer a block of data between two data-memory locations and verify the contents using the file-register window.

```
PROCESSOR 18F4550
#include <xc.in>

MOVLW 0x11
MOVWF 0x20

MOVLW 0x22
MOVWF 0x21H

MOVLW 0x33
```

```
MOVWF 0x22

MOVLW 0x44
MOVWF 0x23

MOVLW 0x55
MOVWF 0x24H

LFSR 0, 0x020   //Load File Select Register. indirect memory addressing.
LFSR 1, 0x030

MOVLW 0x05
MOVWF 0x40H

BACK:
        MOVFF POSTINC0, POSTINC1
        DECFSZ 0x40, F
        BRA BACK

HERE:
         BRA HERE

END
```

Q4) Using the PIC18F4550 microcontroller, configure a timer and toggle an LED periodically using the generated timer delay.

```
#include <xc.h>
#define _XTAL_FREQ 4000000
void main()
{
   TRISB = 0xDF;   //controls whether each pin is input or output
   LATB = 0x00;
   T0CON = 0x07; timer off and prescaler 256 (last 3 bits)
   while(1)
   {
      TMR0H = 0xF8;
      TMR0L = 0x5F; //calculations for 500 ms
      INTCONbits.TMR0IF = 0; //interrupt control flag timer 0 overflow flag is cleared
      T0CON = 0x87;
      while(INTCONbits.TMR0IF == 0); //polling method
      T0CON = 0x07;
      LATBbits.LATB5 = !LATBbits.LATB5;
```

```
    }
}
```

Q5) Using the PIC18F4550 microcontroller, interface eight LEDs through PORTB and two push buttons. Increment the 8-bit count on one button press and decrement it on the other, and display the updated binary count on the LEDs.

```
#include <xc.h>
#define _XTAL_FREQ 4000000
void main()
{
   unsigned char count = 0;
   ADCON1 = 0x0F;
   TRISB = 0x00;
   TRISA = 0x03;
   LATB = 0x00;
   while(1)
   {
      if(PORTAbits.RA0 == 0)
      {
         __delay_ms(20);

         if(PORTAbits.RA0 == 0)
         {
            count++;
            LATB = count;

            while(PORTAbits.RA0 == 0);
         }
      }

      if(PORTAbits.RA1 == 0)
      {
         __delay_ms(20);

         if(PORTAbits.RA1 == 0)
         {
            count--;
            LATB = count;

            while(PORTAbits.RA1 == 0);
         }
      }
   }
```

}

Q6) Using the PIC18F4550 microcontroller, interface eight LEDs and two push buttons. Use one button to start an 8-bit up-counter and the other button to reset the count to 00H. Display the count on the LEDs.

```
#include <xc.h>
#define _XTAL_FREQ 4000000
void main()
{
   unsigned char count = 0;
   unsigned char start= 0;
   ADCON1 = 0x0F;
   TRISB = 0x00;
   TRISA = 0x03;
   LATB = 0x00;
   while(1)
   {
      if(PORTAbits.RA0 == 0)
      {
         __delay_ms(20);

         if(PORTAbits.RA0 == 0)
         {
            start =1
            while(PORTAbits.RA0 == 0);
         }
      }

      if(PORTAbits.RA1 == 0)
      {
         __delay_ms(20);

         if(PORTAbits.RA1 == 0)
         {
            count = 0;
            start = 0;
            LATB = count;

            while(PORTAbits.RA1 == 0);
         }
      }

      if(start == 1)
```

```
        {
            LATB = count;
            __delay_ms(100);
            count++;
        }
    }
}
```

Q7) Using the PIC18F4550 microcontroller, interface eight LEDs and two push buttons. Use one button to shift a single glowing LED from LSB to MSB and the other button to shift it from MSB to LSB.

```
#include <xc.h>
#define _XTAL_FREQ 4000000
void main()
{
    unsigned char led = 0x01;
    ADCON1 = 0x0F;
    TRISB = 0x00;
    TRISA = 0x03;
    LATB = led;

    while(1)
    {
        if(PORTAbits.RA0 == 0)
        {
            __delay_ms(20);
            if(PORTAbits.RA0 == 0)
            {
                if(led == 0x80)
                    led = 0x01;
                else
                    led = led << 1;
                LATB = led;
                while(PORTAbits.RA0 == 0);
            }
        }
        if(PORTAbits.RA1 == 0)
        {
            __delay_ms(20);
            if(PORTAbits.RA1 == 0)
            {
                if(led == 0x01)
                    led = 0x80;
```

```
            else
                led = led >> 1;
            LATB = led;
            while(PORTAbits.RA1 == 0);
        }
    }
  }
}
```

Q8) Using the PIC18F4550 microcontroller, interface a 16x2 LCD and display your name on the first line and roll number on the second line.

```
#include <xc.h>
#define _XTAL_FREQ 4000000
#define RS LATCbits.LATC0
#define EN LATCbits.LATC1

void lcd_cmd(unsigned char cmd)
{
  RS = 0;
  LATD = cmd;

  EN = 1;
  __delay_ms(1);
  EN = 0;
  __delay_ms(2);
}

void lcd_data(unsigned char data)
{
  RS = 1;
  LATD = data;

  EN = 1;
  __delay_ms(1);
  EN = 0;
  __delay_ms(2);
}

void lcd_string(const char *str)
{
  while(*str)
  {
    lcd_data(*str++);
```

```
    }
}

void lcd_init()
{
    __delay_ms(20);

    lcd_cmd(0x38);
    lcd_cmd(0x0C);
    lcd_cmd(0x06);
    lcd_cmd(0x01);
}

void main()
{
    ADCON1 = 0x0F;
    TRISD = 0x00;
    TRISC = 0xFC;
    LATD = 0x00;
    LATC = 0x00;
    lcd_init();
    lcd_cmd(0x80);
    lcd_string("YOUR NAME");
    lcd_cmd(0xC0);
    lcd_string("2024200030");
    while(1);
}
```

Q9) Using the PIC18F4550 microcontroller, interface a 16x2 LCD and display the string 'MICROCONTROLLER' on the first line and 'LAB ESE' on the second line.

Q10) Using the PIC18F4550 microcontroller, interface a 16x2 LCD and continuously display an increasing decimal count from 00 to 99 on the second line while keeping 'COUNT' on the first line.

```
#include <xc.h>

#define _XTAL_FREQ 4000000

#define RS LATCbits.LATC0
#define EN LATCbits.LATC1

void lcd_cmd(unsigned char cmd)
```

```
{
   RS = 0;
   LATD = cmd;

   EN = 1;
   __delay_ms(1);
   EN = 0;
   __delay_ms(2);
}

void lcd_data(unsigned char data)
{
   RS = 1;
   LATD = data;

   EN = 1;
   __delay_ms(1);
   EN = 0;
   __delay_ms(2);
}

void lcd_string(const char *str)
{
   while(*str)
   {
      lcd_data(*str++);
   }
}

void lcd_init()
{
   __delay_ms(20);

   lcd_cmd(0x38);
   lcd_cmd(0x38);
   lcd_cmd(0x0C);
   lcd_cmd(0x06);
   lcd_cmd(0x01);
}

void main()
{
   unsigned char count = 0;
```

```
    ADCON1 = 0x0F;

    TRISD = 0x00;
    TRISC = 0xFC;

    LATD = 0x00;
    LATC = 0x00;

    lcd_init();

    lcd_cmd(0x80);
    lcd_string("COUNT");

    while(1)
    {
        lcd_cmd(0xC0);

        lcd_data((count / 10) + '0');
        lcd_data((count % 10) + '0');

        __delay_ms(500);

        count++;

        if(count > 99)
        {
            count = 0;
        }
    }
}
```

Q11) Using the PIC18F4550 microcontroller, interface a stepper motor through the available driver circuit and rotate the motor continuously in the clockwise direction using the correct stepping sequence.

```
#include <xc.h>

void delay(unsigned int step)
{
    unsigned int i, j;

    for(i = 0; i < step; i++)
    {
```

```
        for(j = 0; j < 1275; j++);
    }
}

void main(void)
{
    unsigned int i;

    unsigned char stepper_pattern[] =
    {0x01, 0x03, 0x02, 0x06,
     0x04, 0x24, 0x20, 0x21};

    ADCON1 = 0x0F;
    PORTC = 0x00;
    TRISC = 0x00;

    while(1)
    {
        for(i = 0; i < 8; i++)
        {
            PORTC = stepper_pattern[i];
            delay(5);
        }
    }
}
```

Q12) Using the PIC18F4550 microcontroller, interface a stepper motor and rotate it continuously in the anticlockwise direction using the correct stepping sequence.

```
#include <xc.h>

void delay(unsigned int step)
{
    volatile unsigned int i, j;

    for(i = 0; i < step; i++)
    {
        for(j = 0; j < 1275; j++);
    }
}

void main(void)
{
```

```
    unsigned int i;

    unsigned char stepper_pattern[] =
    {0x21, 0x20, 0x24, 0x04,
     0x06, 0x02, 0x03, 0x01};

    ADCON1 = 0x0F;
    PORTC = 0x00;
    TRISC = 0x00;

    while(1)
    {
        for(i = 0; i < 8; i++)
        {
            PORTC = stepper_pattern[i];
            delay(5);
        }
    }
}
```

Q13) Using the PIC18F4550 microcontroller, interface a stepper motor and two push buttons. Rotate the motor clockwise when the first button is pressed and anticlockwise when the second button is pressed.

```
#include <xc.h>
#define _XTAL_FREQ 4000000
void main()
{
    unsigned char i = 0;
    unsigned char stepper_pattern[] =
    {0x01, 0x03, 0x02, 0x06,
     0x04, 0x24, 0x20, 0x21};
    ADCON1 = 0x0F;
    TRISA = 0x03;
    TRISC = 0x00;
    LATC = 0x00;

    while(1)
    {
        if(PORTAbits.RA0 == 0)
        {
            LATC = stepper_pattern[i];
```

```
            __delay_ms(5);
            i++;
            if(i == 8)
                i = 0;
        }

        else if(PORTAbits.RA1 == 0)
        {
            LATC = stepper_pattern[i];
            __delay_ms(5);
            if(i == 0)
                i = 7;
            else
                i--;
        }
        else
        {
            LATC = 0x00;
        }
    }
}
```

Q14) Using the PIC18F4550 microcontroller, interface a stepper motor and write a program to rotate it one complete revolution clockwise, pause, and then one complete revolution anticlockwise.

```
#include <xc.h>
#define _XTAL_FREQ 4000000

void main()
{
    unsigned int i, j;
    unsigned char stepper_pattern[] =
    {0x01, 0x03, 0x02, 0x06,
     0x04, 0x24, 0x20, 0x21};

    ADCON1 = 0x0F;
    PORTC = 0x00;
    TRISC = 0x00;

    // Clockwise - One revolution
    for(j = 0; j < 50; j++)
    {
```

```
        for(i = 0; i < 8; i++)
        {
            PORTC = stepper_pattern[i];
            __delay_ms(5);
        }
    }

    // Pause for 2 seconds
    __delay_ms(2000);

    // Anticlockwise - One revolution
    for(j = 0; j < 50; j++)
    {
        for(i = 8; i > 0; i--)
        {
            PORTC = stepper_pattern[i - 1];
            __delay_ms(5);
        }
    }

    PORTC = 0x00;

    while(1);
}
```

Q15) Using the 8051 microcontroller, write a program using Timer 0 in polling mode to generate a 1 kHz square wave on an output pin and verify the waveform using a CRO/DSO or Logic Analyzer.

```
#include <reg51.h>

void T0M1Delay(void);
sbit mybit = P1^5;

void main(void)
{
    while(1)
    {
        mybit = ~mybit;
        T0M1Delay();
    }
}

void T0M1Delay(void)
```

```
{
   TMOD = 0x01;
   TL0 = 0x33;
   TH0 = 0xFE;
   TR0 = 1;

   while(TF0 == 0);

   TR0 = 0;
   TF0 = 0;
}
```

Q16) Using the 8051 microcontroller, write a program using Timer 1 in polling mode to generate two frequencies, 500 Hz and 1 kHz, sequentially on the same output pin and verify both frequencies using a CRO/DSO or Logic Analyzer.

```
#include <reg51.h>

sbit mybit = P1^5;
void T1M1Delay(unsigned char c);

void main(void)
{
   unsigned int i;

   while(1)
   {
      for(i=0;i<1000;i++)
      {
         mybit = ~mybit;
         T1M1Delay(0);
      }

      for(i=0;i<2000;i++)
      {
         mybit = ~mybit;
         T1M1Delay(1);
      }
   }
}

void T1M1Delay(unsigned char c)
{
   TMOD = 0x10;
```

```
    if(c==0)
    {
        TL1 = 0x66;
        TH1 = 0xFC;
    }
    else
    {
        TL1 = 0x33;
        TH1 = 0xFE;
    }

    TR1 = 1;
    while(TF1==0);
    TR1 = 0;
    TF1 = 0;
}
```

Q17) Using the 8051 microcontroller, write a program using Timer 0 or Timer 1 in polling mode to generate a hybrid waveform consisting of two cycles of 400 Hz followed by two cycles of 900 Hz, repeating continuously. Verify the waveform using a Logic Analyzer.

```
#include <reg51.h>

sbit mybit = P1^5;
void T0M1Delay(unsigned char c);

void main(void)
{
    unsigned char i;

    mybit = 0;

    while(1)
    {
        for(i=0;i<4;i++)
        {
            mybit = ~mybit;
            T0M1Delay(0);
        }

        for(i=0;i<4;i++)
        {
            mybit = ~mybit;
            T0M1Delay(1);
```

```
      }
   }
}

void T0M1Delay(unsigned char c)
{
   TMOD = 0x01;

   if(c==0)
   {
      TL0 = 0x80;
      TH0 = 0xFB;
   }
   else
   {
      TL0 = 0x00;
      TH0 = 0xFE;
   }

   TR0 = 1;
   while(TF0==0);
   TR0 = 0;
   TF0 = 0;
}
```

Q18) Using the 8051 microcontroller, configure serial communication at the specified baud rate and continuously transmit your name as a complete string to the serial terminal.

```
#include <reg51.h>

void main(void)
{
   unsigned char i;
   unsigned char name[] = "ANUJ";

   TMOD = 0x20;
   TH1 = 0xFD;
   SCON = 0x50;
   TR1 = 1;

   while(1)
   {
      for(i=0;name[i]!='\0';i++)
      {
```

```
            SBUF = name[i];
            while(TI==0);
            TI = 0;
        }
    }
}
```

Q19) Using the 8051 microcontroller, configure serial communication and continuously transmit your name and roll number as a complete string to the serial terminal.

```
#include <reg51.h>

void main(void)
{
    unsigned char i;
    unsigned char name[] = "ANUJ 2024200030\r\n";

    TMOD = 0x20;
    TH1 = 0xFD;
    SCON = 0x50;
    TR1 = 1;

    while(1)
    {
        for(i=0;name[i]!='\0';i++)
        {
            SBUF = name[i];
            while(TI==0);
            TI = 0;
        }
    }
}
```